AI505 – Optimization

Answers to Obligatory Assignment 2, Spring 2025

Daniele Mensorio

1 Optimization for Machine Learning

The methods tried to optimize LeNet5 are the following:

- **Basic SGD** (Stochastic Gradient Descent):

  SGD is a stochastic optimization method, defined by the iteration:

  $$w_{k+1} \leftarrow w_k - \alpha_k \nabla f_{i_k}(w_k) \qquad \forall k \in \mathbb{N} := \{1, 2, \dots\}$$

  where:

  - the index i_k represents the seed (the sample pair (x_{i_k}, y_{i_k}) is chosen randomly from $\{1, 2, \dots, n\}$);
  - α_k denotes the positive step-size;
  - $\{w_k\}$ is a stochastic process whose behavior is determined by the random sequence $\{i_k\}$.

  Each iteration is very cheap, involving the computation of the gradient for one sample.

  SGD uses a noise gradient approximation (add noise to each descent step, that is reduced over time) and, basically, evaluates noisy gradients using randomly chosen batches (subset of the training data), resulting less expensive than calculating the true gradient at every iteration.

  Stochastic optimization methods are more efficient than batch one, since stochastic gradient computation involves choosing elements from subsets of the training data with the same probability, implying that, with the batch method, each optimization iteration, employing all part of the sample data, would be more expensive. So, working with small samples can improve performance.

  Algorithm 1: Stochastic Gradient (SG) Method

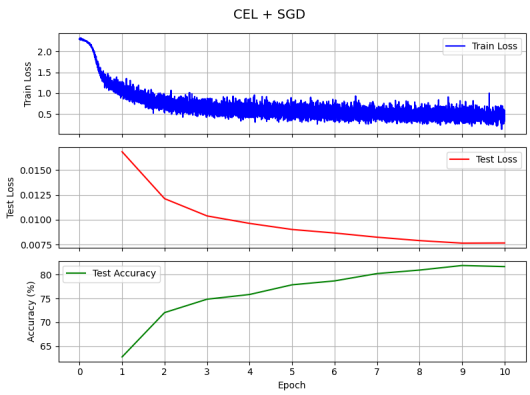
  Input: Initial iterate w_1;
  for $k = 1, 2, \dots$ **do**
  Generate a realization of the random variable ξ_k;
  Compute a stochastic gradient $g(w_k, \xi_k)$:

  $$g(w_k, \xi_k) = \nabla f(w_k; \xi_k)$$

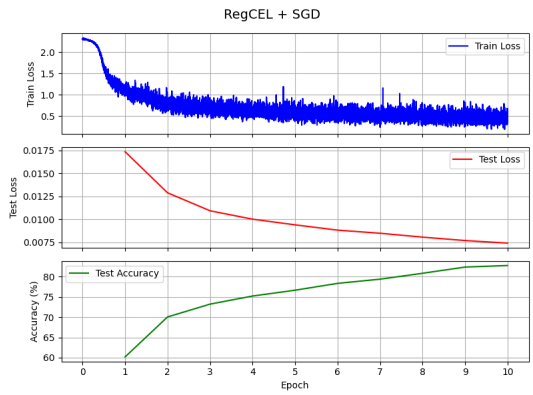
  Choose a step-size $\alpha_k > 0$;
  Set $w_{k+1} = w_k - \alpha_k \nabla f_{i_k}(w_k)$;

  In the general algorithm 1 for SGD:

- the random variable ξ_k is a seed for generating a stochastic direction: a realization of it may represent the the choice of a single training sample: i.e. sample pair (x_{i_k}, y_{i_k}) chosen randomly from $\{1, 2, \ldots, n\}$;
- $g(w_k, \xi_k)$ identifies a stochastic gradient;
- the step-size sequence $\{\alpha_k\}$ can be subject to different choices: fixed or diminishing step-size.



(a) CEL SGD



(b) RegCEL SGD

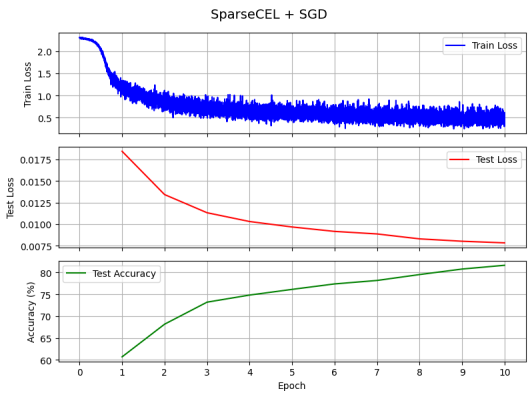


Figure 1
SparseCEL SGD

Overall, traditional SGD performs the worst. Particularly under SparseCEL, where the additional instability from sparse weights increases the variance of its updates, its convergence is more irregular and slower. The noise in SGD's updates may be made worse by the extra complexity that sparsity requirements provide. When compared to CEL and RegCEL, the learning curve is less slick.

Figure 2: SGD performance across three loss functions.

- **Mini-batch SGD**;

  Mini-batch SGD is neither purely stochastic not purely batch, but attempt to combine the best property of the two approaches, by choosing randomly a small subset of samples, $S_k \subseteq \{1, 2, \ldots, n\}$ in each iteration:

$$w_{k+1} \leftarrow w_k - \frac{\alpha_k}{|S_k|} \sum_{i \in S_k} \nabla f_i(w_k)$$

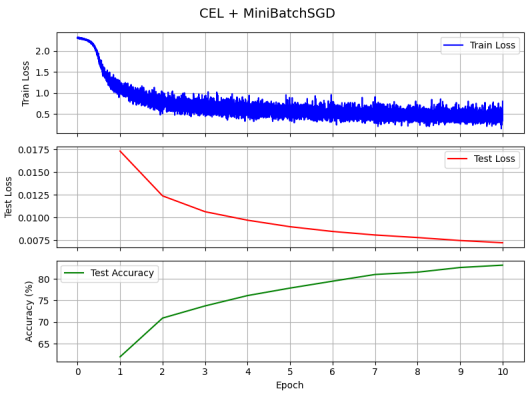
  An advantage of this approach is the degree of parallelization due to the computation of mini-batch gradients and the lower variance of the stochastic gradient estimates that make it easy to tune in terms of choosing the step-sizes $\{\alpha_k\}$.

Algorithm 2: Mini-batch Stochastic Gradient Method

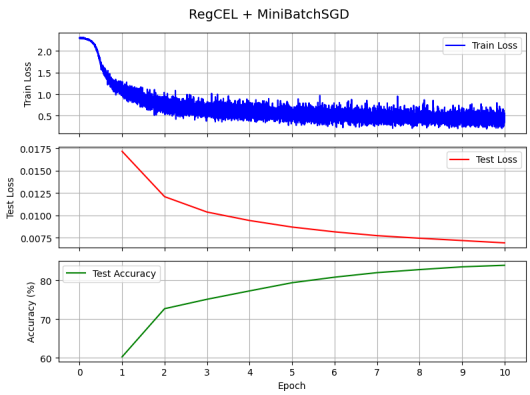
Input: Initial iterate w_1, mini-batch size n_k;
for $k = 1, 2, \ldots$ **do**
  Sample a mini-batch $\{\xi_{k,1}, \ldots, \xi_{k,n_k}\}$;
  Compute stochastic gradient:

$$g(w_k, \xi_k) = \frac{1}{n_k} \sum_{i=1}^{n_k} \nabla f(w_k; \xi_{k,i})$$

  Choose step-size $\alpha_k > 0$;
  Update: $w_{k+1} = w_k - \alpha_k g(w_k)$;



(a) CEL Mini-Batch SGD



(b) RegCEL Mini-Batch SGD

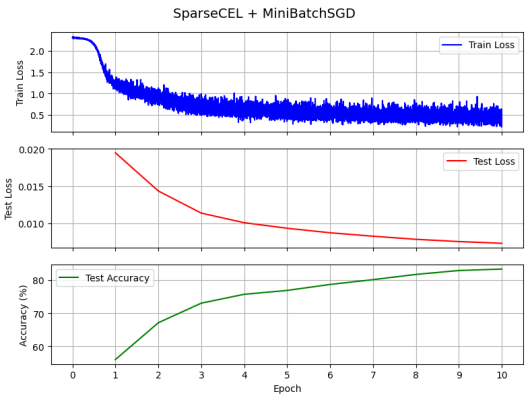


Figure 3
SparseCEL Mini-Batch SGD

Through averaging, mini-batch SGD reduces gradient noise, improving stability across all loss types and final accuracy when compared to pure SGD. Although there are still some oscillations with SparseCEL, the variance is noticeably lower. This makes it a sensible option when weighing noise reduction against computational expense, particularly when sparsity is present.

Figure 4: Mini-batch SGD performance across CEL, RegCEL, and SparseCEL.

- **Batch SGD**: Steepest descent:

  Steepest descent is a batch optimization method, defined by the iteration:

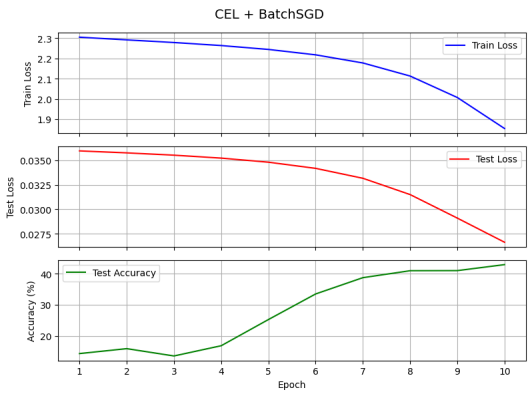
$$w_{k+1} \leftarrow w_k - \alpha_k \nabla R_n(w_k)$$
$$w_{k+1} \leftarrow w_k - \frac{\alpha_k}{n} \sum_{i=1}^{n} \nabla f_i(w_k)$$

Each iteration is more expensive with respect to a stochastic approach since the computation of the gradient is more complex. A benefit of batch methods is the possibility of parallelization due to the sum structure of R_n and since most of the computations involve the evaluation of R_n and ∇R_n, operations that can be easily distributed.

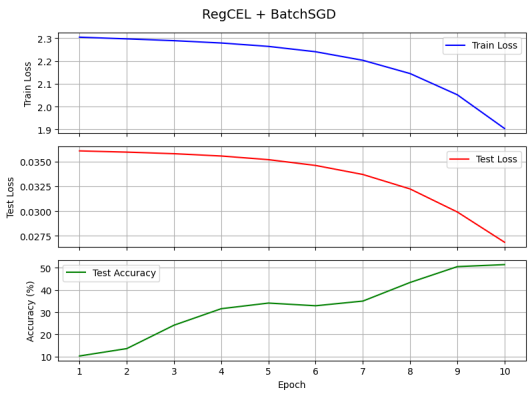
Algorithm 3: Steepest Descent

Input: f, ϵ, $\nabla f(x)$
Output: x^*

```
Set counter k ← 0, g^(k) ← ∇f(x^(k));
Choose a starting point x^(0);
while ||g^(k)|| < ε do
    d^(k) ← −g^(k)^T;
    α^(k) ← arg min_α f(x^(k) + α d^(k));
    x^(k+1) ← x^(k) + α^(k) d^(k);
    k ← k + 1;
return x^(k)
```



(a) CEL Batch SGD



(b) RegCEL Batch SGD

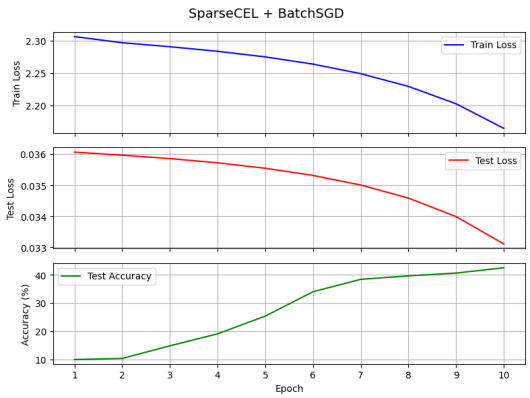


Figure 5
SparseCEL Batch SGD

Because batch SGD computes gradients for the entire dataset, it offers smooth and consistent convergence for CEL and RegCEL, but at the expense of slower per-epoch progress. Although the convergence time is slower, it is still more reliable than simple SGD under SparseCEL. This implies that batch approaches prefer loss surfaces that are organized and quieter.

Figure 6: Batch SGD performance across CEL, RegCEL, and SparseCEL.

- **SGD with momentum**:

  SGD with momentum is a procedure in which each step is chosen as a combination of the steepest descent direction and the most recent iterate displacement. The addition

of momentum allows to speed progress to traverse nearly flat surface. Each iteration, with an initial point w_1 and $w_0 := w_1$, is characterized by the following:

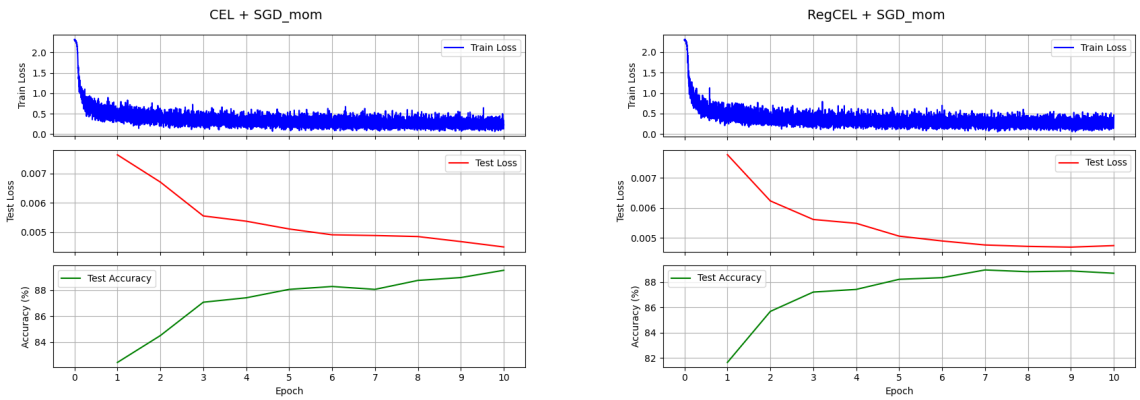
$$w_{k+1} \leftarrow w_k - \alpha_k \nabla f_{i_k}(w_k) + \beta_k(w_k - w_{k-1})$$

where $\{\alpha_k\}$ and $\{\beta_k\}$ are scalar sequences, either predetermined or dynamically set and $\beta_k(w_k - w_{k-1})$, the momentum term, recursively maintains the algorithm's movement along previous search directions.

The momentum can be interpreted as a ball rolling down a nearly horizontal incline: the ball naturally gathers momentum as gravity causes it to accelerate, just as the gradient causes momentum to accumulate in the descent direction. For this reason, when $\alpha_k = \alpha$ and $\beta_k = \beta$ for some $\alpha > 0$ and $\beta > 0$ for all $k \in \mathbb{N}$ this method is also known as the heavy-ball method, which can also be defined as:

$$w_{k+1} \leftarrow w_k - \alpha \sum_{j=1}^{k} \beta^{k-j} \nabla f_{i_k}(w_k)$$

Each iteration, over the long run, tends to accumulate contributions in the direction of persistent descent and moves in directions that the stochastic gradients suggest are ones of improvement.



(a) CEL SGD with momentum

(b) RegCEL SGD with momentum

2

Figure 7: SGD with momentum performance across CEL, RegCEL, and SparseCEL.

- **SGD with Nesterov momentum**:

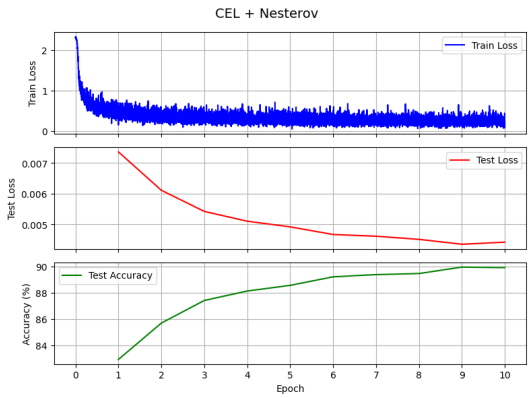
  SGD with Nesterov momentum works similarly as the previous method but it is a two-step procedure, involving two updates:

$$\tilde{w}_k \leftarrow w_k + \beta_k(w_k - w_{k-1})$$
$$w_{k+1} \leftarrow \tilde{w}_k - \alpha_k \nabla f_{i_k} \tilde{w}_k$$

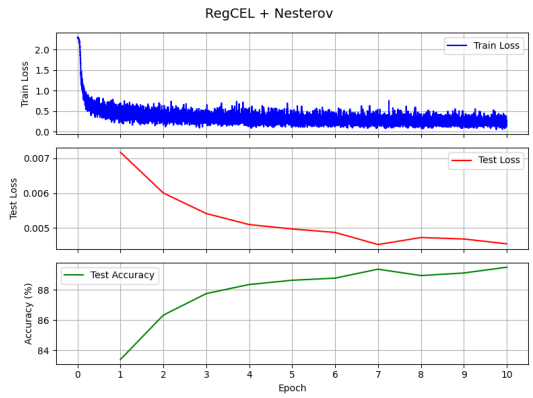
  which is equal to:

$$w_{k+1} \leftarrow w_k - \alpha_k \nabla f_{i_k}(w_k + \beta_k(w_k - w_{k-1})) + \beta_k(w_k - w_{k-1})$$

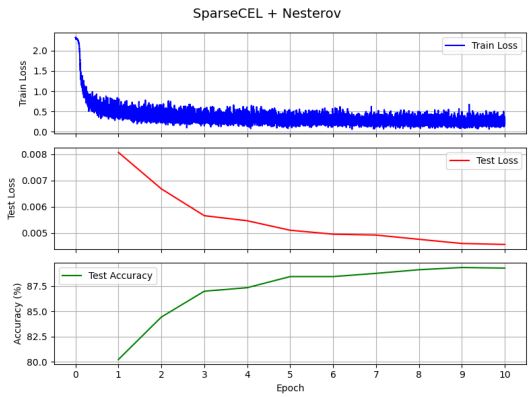
The classical momentum method first takes the steepest descent step and then applies the momentum term; instead, the Accelerated gradient method proposed by Nesterov first follows the momentum term and then applies the steepest descent, with the gradient evaluated at $\tilde{w}_k$ not at w_k. The Nesterov approach, proposed because one issue of the momentum is that it does not tend to slow down enough at the bottom of the valley and tends to overshoot the valley floor, is to update the momentum by speed by calculating the gradient not on the current point but on the point ahead in order to see if the gradient will stay stronger or decrease.



(a) CEL SGD with Nesterov momentum



(b) RegCEL SGD with Nesterov momentum



(c) SparseCEL SGD with Nesterov momentum

Nesterov Accelerated Gradient exhibits a competitive advantage, particularly in contrast to conventional momentum-based SGD. Both the CEL and RegCEL settings benefit from the smoother and faster convergence made possible by Nesterov's preemptive upgrade. Nesterov still has some advantages under SparseCEL, but the benefit is less noticeable—perhaps as a result of the increased noise and instability brought on by sparsity constraints.

Figure 8: SGD with Nesterov momentum perfomance across CEL, RegCEL and SparseCEL

- **Natural Gradient Approximation**: a gradient iteration method;

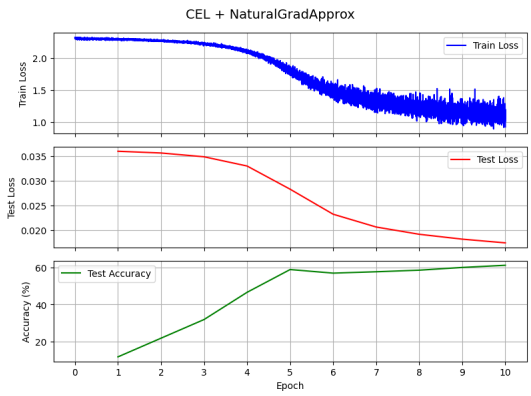
  Natural Gradient Approximation is a second-order method that defines a search direction in the space of realizable distributions (in the space of prediction functions), rather than in the space of the real parameter vector w. Whereas Newton's method is invariant for linear transformations of variables, the natural gradient method is invariant with respect to more general invertible transformations. That is, in parameter space, the natural gradient algorithm will move the parameters more quickly along directions that have a small impact on the decision function, and more cautiously along directions that have a large impact on the decision function.

  Each iteration is defined by the following:

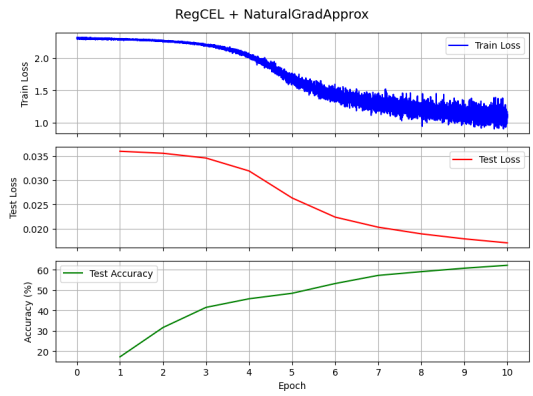
$$w_{k+1} \leftarrow w_k - \alpha_k G^{-1}(w_k) \nabla F(w_k)$$

where:

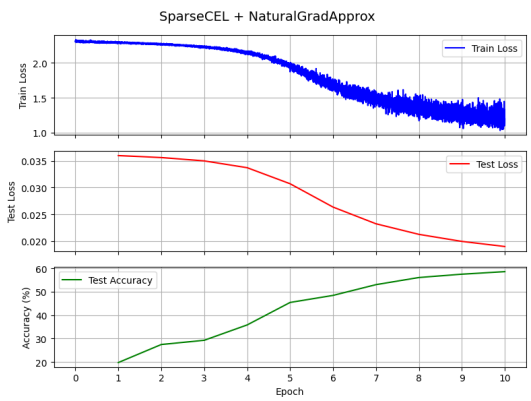
- $F(w)$ is a noise first-order approximation and $\nabla F(w_k)$ represents the stochastic gradient estimate;
- the positive definite matrix $G(w_k)$ (the Fisher information matrix) express the local geometry of the space prediction functions.



(a) CEL SGD with Natural Gradient Approximation



(b) RegCEL SGD with Natural Gradient Approximation



(c) SparseCEL SGD with Natural Gradient Approximation

Natural Gradient Approximation provides smooth learning curves and stable convergence for CEL and RegCEL, benefiting from more accurate updates. While it's slower compared to other optimizers, it is useful when curvature-informed updates are needed without full second-order calculations. SparseCEL shows less benefit, likely due to challenges with sparsity constraints.

Figure 9: SGD with Natural Gradient Approximation performance across CEL, RegCEL, and SparseCEL

- **LBFGS** (Limited BFGS): a quasi-newton method:

  The limited BFGS (Broyden-Fletcher-Goldfarb-Shanno) method is a second-order method: a quasi-netwon method approximates the inverse Hessian using only gradient information with this update:

$$w_{k+1} \leftarrow w_k - \alpha_k H_k \nabla f_{i_k}(w_k)$$

  for stochastic optimization, it evolves in:

$$w_{k+1} \leftarrow w_k - \alpha_k H_k g(w_k, \xi_k)$$

  where :

- α_k is a scalar step factor;
- H_k approximate the inverse of the Hessian at w_k and it is a symmetric positive definite approximation of $(\nabla^2 f_{i_k}(w_k))^{-1}$.

H_k is set to the identity matrix and then applies updates to reflect the information learned with each iteration. The new inverse Hessian approximation is obtained by defining the iterate and gradient displacements like the following:

$$
\begin{aligned}
v_k &:= \begin{cases} \text{batch:} \quad \nabla f_{i_k}(w_{k+1}) - \nabla f_{i_k}(w_k) \\ \text{stochastic:} \quad \nabla f_{S_k}(w_{k+1,\xi_{k+1}}) - \nabla f_{S_k}(w_k, \xi_k) \end{cases} \\
s_k &:= w_{k+1} - w_k
\end{aligned}
$$

L-BFGS method sets:

$$
H_{k+1} \leftarrow \left(I - \frac{v_k s_k^T}{s_k^T v_k} \right)^T H_k \left(I - \frac{v_k s_k^T}{s_k^T v_k} \right) + \frac{s_k s_k^T}{s_k^T v_k}
$$

L-BFGS stores the last m values for s and v rather than the full inverse Hessian.

The matrices $\{H_k\}$ need not be formed explicitly; instead, each product of the form $H_k \nabla f_{i_k}(w_k)$ can be computed using a formula that only requires recent elements of the sequence of displacement pairs $\{(s_k, v_k)\}$ that have been saved in storage in order to incur per-iteration costs.

- **Iterate Averaging method**: a noise reduction techniques;

  Iterate averaging method is a noise reduction techniques that accomplish noise reduction not by averaging gradient estimates, but by maintaining an average of iterates computed during the optimization process.

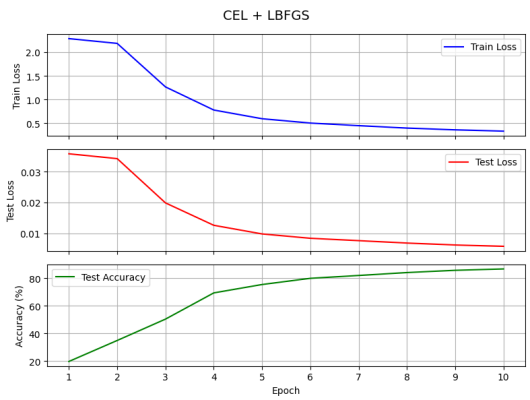
  This method computes a corresponding sequence of iterate averages that would automatically possess less noisy behavior. The iteration is defined by the following step:

$$
\begin{aligned}
w_{k+1} &\leftarrow w_k - \alpha_k g(w_k, \xi_k) \\
\tilde{w}_{k+1} &\leftarrow \frac{1}{k+1} \sum_{j=1}^{k+1} w_j
\end{aligned}
$$

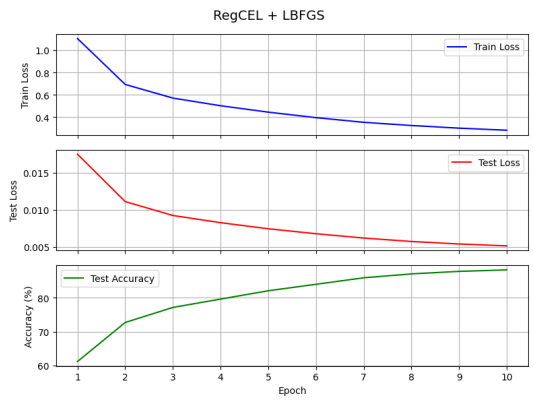
  where the averaged sequence $\{\tilde{w}_k\}$ has no effect on the computation of the SG iterate sequences $\{w_k\}$.

- **Adam** (an adaptive moment estimation method):

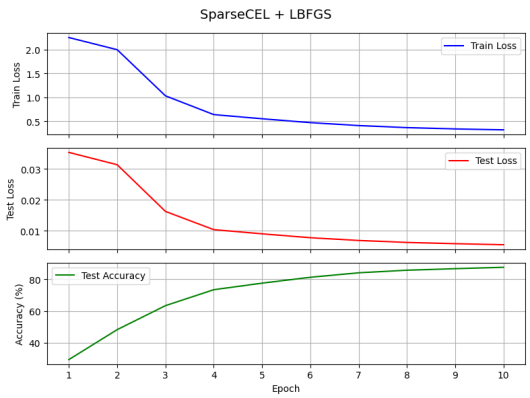
  Adam adapts the learning rates to each parameter: it stores both an exponentially decaying squared gradient and also an exponentially decaying gradient, like momentum. However,m initializing the gradient and the squared gradient to zero introduces bias and for that reason, a bias correction step is used in order to alleviate the issue. In each iteration, these following steps are taken:



(a) CEL L-BFGS



(b) RegCEL L-BFGS



(c) SparseCEL L-BFGS

In the CEL and RegCEL cases in particular, L-BFGS exhibits a relatively quick initial decline. This is in line with its quasi-Newton updates and its design to approximate second-order curvature information. Its performance seems more erratic in the SparseCEL environment, though, indicating that the assumptions underlying L-BFGS could not be compatible with the discontinuities or flat areas brought about by imposed sparsity. It performs better in situations requiring exact curvature information, although being slower than techniques like SGD.

Figure 10: Performance of L-BFGS across CEL, RegCEL, and SparseCEL

biased decaying momentum: $v^{k+1} = \gamma_s v^k + (1-\gamma_v) g^{(k)}$

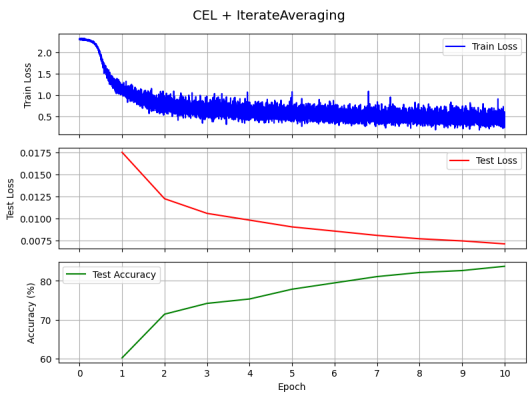
biased decaying squared gradient: $s^{(k+1)} = \gamma_s s^{(k)} + (1-\gamma_s)(\nabla f_{i_k} w_k \odot \nabla f_{i_k} w_k)$

corrected decaying momentum: $\hat{v}^{k+1} = \dfrac{v^{k+1}}{(1-\gamma_v^{(k)})}$

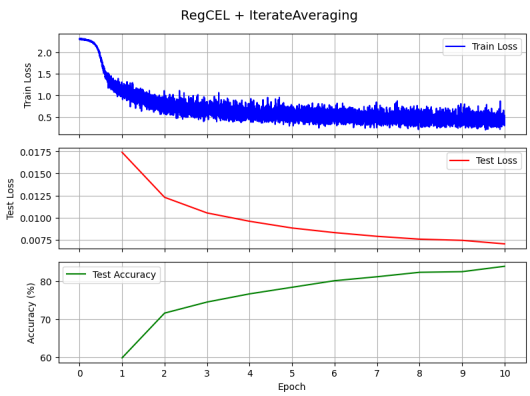
corrected decaying squared gradient: $\hat{s}^{k+1} = \dfrac{s^{k+1}}{(1-\gamma_s^{(k)})}$

next iterate: $w^{(k+1)} = w^{(k)} - \dfrac{\alpha \hat{v}^{(k+1)}}{\epsilon + \sqrt{\hat{s}^{(k+1)}}}$

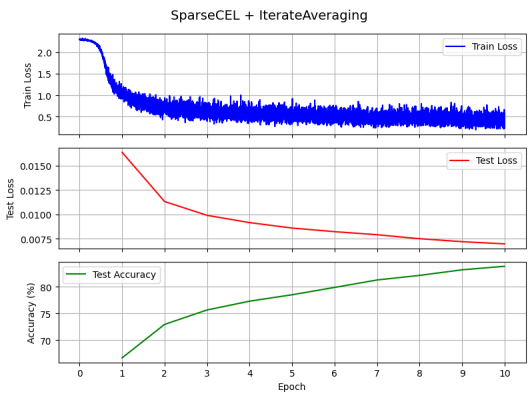
so, the iterations is defined by:



(a) CEL SGD with Iterate Averaging



(b) RegCEL SGD with Iterate Averaging



(c) SparseCEL SGD with Iterate Averaging

Under sparseCEL, iterate averaging works well because it smooths the optimization trajectory and reduces noise by averaging model parameters throughout training. This makes it especially useful when the loss surface is jagged, as it is under sparsity constraints. Under such circumstances, this approach can stabilize learning.

Figure 11: Performance of SGD with Iterate Averaging across CEL, RegCEL, and SparseCEL

$$w_{k+1} \leftarrow w_k - \frac{\alpha \frac{\gamma_s v_k + (1-\gamma_v)\nabla f_{i_k}(w_k)}{(1-\gamma_v)}}{\epsilon + \sqrt{\frac{\gamma_s s_k + (1-\gamma_s)(\nabla f_{i_k}(w_k) \odot \nabla f_{i_k}(w_k))}{(1-\gamma_s)}}}$$

$$w_{k+1} \leftarrow w_k - \frac{\alpha(\gamma_s v_k + (1-\gamma_v)\nabla f_{i_k}(w_k))}{(1-\gamma_v)\left(\epsilon + \sqrt{\frac{\gamma_s s_k + (1-\gamma_s)(\nabla f_{i_k}(w_k) \odot \nabla f_{i_k}(w_k))}{(1-\gamma_s)}}\right)}$$

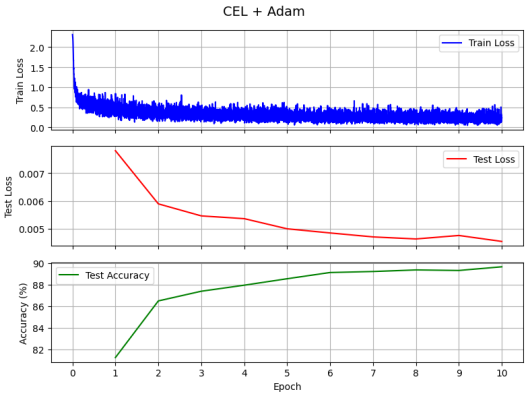
where:

- γ is a decay value, $\gamma \in [0, 1]$;
- $s_i^{(k)}$ is a vector whose its entry is the sum of squares of the partials, with respect to w_i, up to the time step k:

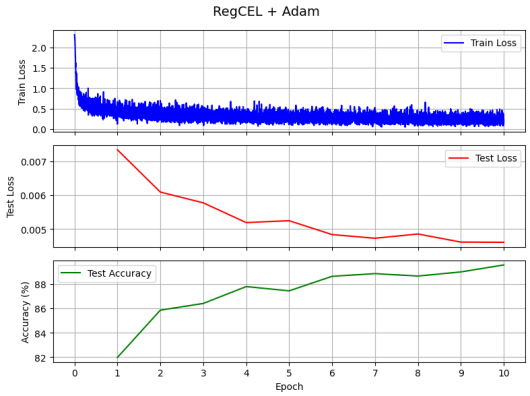
$$s_i^{(k)} = \sum_{j=1}^{k} \left(g_i^{(j)}\right)^2$$

  where $g = \nabla f_{i_k} w_k$;
- ϵ is a small value of order of 1×10^{-8} to prevent division by zero.

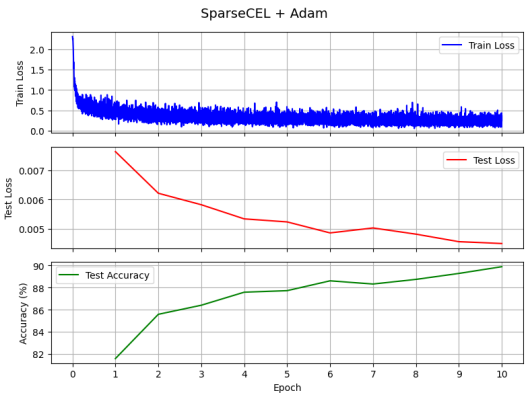
So, Adam will take large steps when gradients are small, smaller steps when gradients are large. Whereas, as we have seen before, momentum can be seen as a ball running down a slope, Adam behaves like a heavy ball with friction, which thus prefers flat minima in the error surface.



(a) CEL Adam



(b) RegCEL Adam



(c) SparseCEL Adam

Adam performs well in both CEL and RegCEL and is effective for large-scale optimization. It provides faster convergence by combining adaptive learning rates with momentum. Perhaps as a result of the noise and instability caused by sparsity limitations, SparseCEL performs worse.

Figure 12: Performance of Adam across CEL, RegCEL, and SparseCEL

2 Conclusion

We start with the optimization techniques and find that L-BFGS, Adam, and Nesterov consistently perform well across the majority of loss criteria. During training, Adam, in particular, is notable for its high test accuracy and quick convergence. Effective learning even in the early epochs is suggested by the Adam curves in the CEL and RegCEL, which show this the most. It works well in a variety of loss conditions, probably due to its adjustable learning rate mechanism, which enables it to efficiently negotiate a wide range of curvatures.

Now, Let us compare information about the loss functions across optimizers. Overall, CEL (Cross Entropy Loss) does well, especially when combined with more sophisticated optimizers such as Adam, L-BFGS, or Nesterov. The majority of optimizers are well-suited to this goal, as seen by the learning curves being generally quick and steady. Under SparseCel (Sparse Weight Constraint), a number of optimizers exhibit erratic or extremely noisy convergence behaviors. However, techniques that use noise reduction techniques, such as

Adam and Iterate Averaging, show that they can adjust to the more challenging environment that sparsity brings. Conversely, techniques such as L-BFGS or SGD seem to have trouble, probably because they depend on continuity and smoothness assumptions that sparsity breaks.

The comparison study shows that Adam is the most consistent optimizer in all three categories, providing accuracy and speed without requiring much tweaking. When adaptive methods, like Adam, are not preferred, Nesterov Accelerated Gradient is an excellent substitute because it strikes a nice balance between stability and convergence speed. Iterate averaging is a useful technique for sparsity-enforced models since it exhibits clear value under sparse restrictions. RegCEL (Regularized Cross-Entropy Loss) seems to be the most optimizer-friendly in terms of loss functions, providing training dynamics that are more constant and smooth. Although SparseCEL requires more careful optimization and benefits from techniques that can limit noise and instability, CEL is still a solid foundation, especially when combined with powerful optimizers. The ideal combination ultimately depends on the particular use case; Adam + RegCEL is a great option if training effectiveness and performance are top concerns. SparseCEL combined with iterate averaging might be the most reliable set-up if model interpretability or sparsity are crucial.